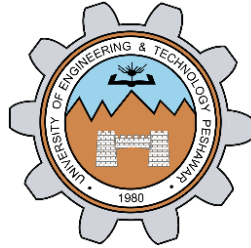


Lab# 02: Passing and Returning Objects from Member Functions



Fall 2025

Submitted by
M Mohsin Khan (24pwcse2382)

Class Section: C

"On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work."

Submitted to: **Engr. Sumayyah Salahuddin**

Oct 23th, 2025

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

2.1 Introduction

This lab aims to provide a comprehensive understanding of class objects in C++ and their use as function arguments and return values in member functions.

It also focuses on developing the ability to test member functions effectively using well-defined test cases.

2.2 Objectives of the Lab

By the end of this lab, students will be able to:

1. Understand how class objects can be passed to and returned from member functions.
2. Write a class that includes member functions accepting objects as parameters.
3. Implement member functions that return objects.
4. Test these member functions effectively using provided or custom test cases.

2.3 Pre-Lab Preparation

2.3.1 Objects as Function Arguments

1. A member function has *direct access* to all members (both public and private) of its own class.
2. A function can also have *indirect access* to members of another object of the same class when that object is passed as an argument.

Indirect Access:

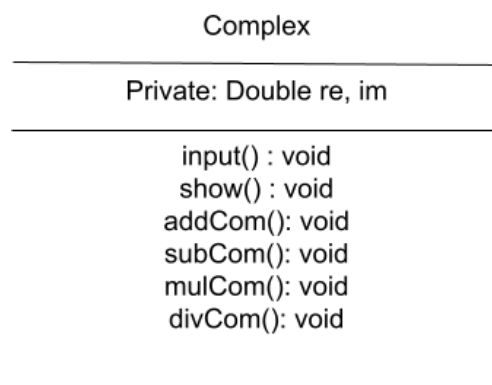
This is achieved using the object name, followed by a dot (.), and then the member name.

2.4.1 Activity

Reuse Complex class written in Activity 2.4.1 to modify the addCom(), subCom(), and mulCom() class functions. Instead of passing two objects in each, now pass only one object.

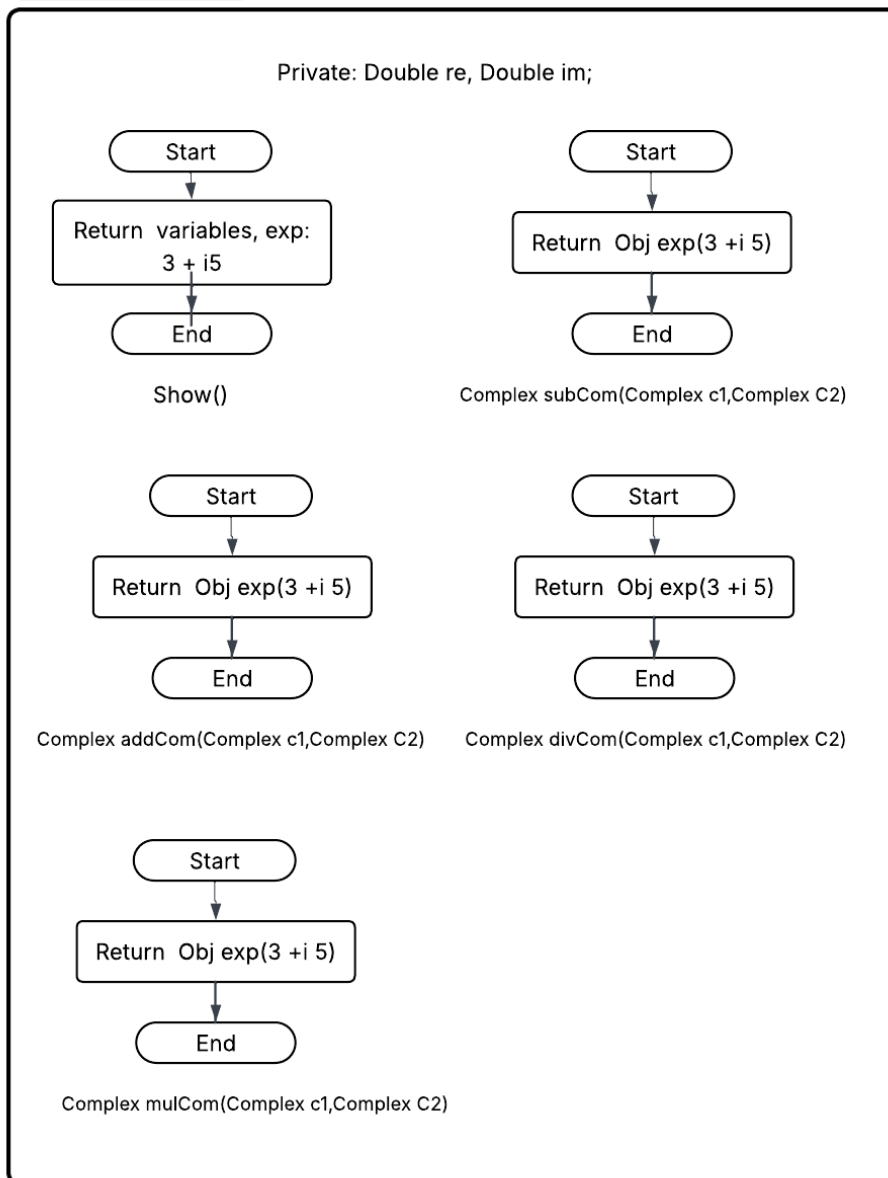
Change the return type of each function to complex. Adjust the function code to match the changes.

2.5.1 ML Diagram:

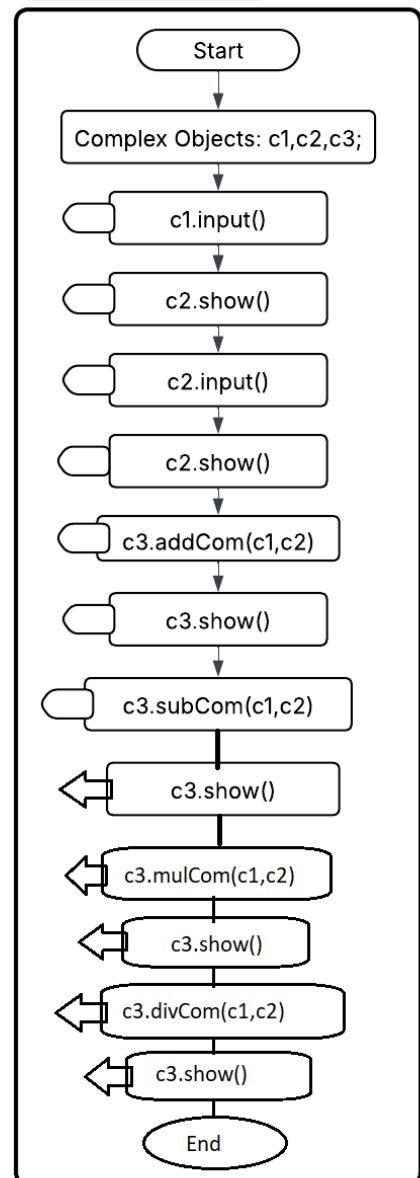


2.5.2 Flow Chart

Class Complex



Function main()



HandWritten Code:

Lab #02

Day:

```
#include <bits/stdc++.h>
using namespace std;
class Complex {
private:
    double re, im;
public:
    Complex(): re(0.0), im(0.0) {}
    Complex(double r, double i): re(r), im(i) {}

    void input() {
        cout << "Enter real part: ";
        cin >> re;
        cout << "Enter imaginary part: ";
        cin >> im;
    }

    void show() {
        if (im > 0)
            cout << re << " + i " << fabs(im) << endl;
        else cout << re << " - i " << fabs(im) << endl;
    }

    void subCom(Complex c1, Complex c2);
    void addCom(Complex c1, Complex c2);
    void mulCom(Complex c1, Complex c2);
};
```

Date:

Day:

```
void Complex::subCom(Complex c1, Complex c2) {
    re = c1.re - c2.re;
    im = c1.im - c2.im;
}

void Complex::addCom(Complex c1, Complex c2) {
    re = c1.re + c2.re;
    im = c1.im + c2.im;
}

void Complex::mulCom(Complex c1, Complex c2) {
    re = (c1.re * c2.re) - (c1.im * c2.im);
    im = (c1.re * c2.im) + (c1.im * c2.re);
}

int main() {
    Complex c1, c2, c3;
    cout << "Enter first Complex number: " << endl;
    c1.input(); c1.show();
    cout << "Enter second Complex number: " << endl;
    c2.input(); c2.show();
    cout << "---Results---" << endl;
    c3.addCom(c1, c2); cout << "Addition" << endl;
    c3.show();
    c3.subCom(c1, c2); cout << "Subtraction" << endl;
    c3.show();
    c3.mulCom(c1, c2);
    cout << "Multiplication" << endl;
    c3.show();
    return 0;
}
```

2.5.1 Source Code:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Complex
5  {
6  private:
7      double re, im;
8
9  public:
10     Complex() : re(0.0), im(0.0) {}
11     Complex(double r, double i) : re(r), im(i) {}
12
13     void input()
14     {
15         cout << "Enter real part: ";
16         cin >> re;
17         cout << "Enter imaginary part: ";
18         cin >> im;
19     }
20
21     void show()
22     {
23         if (im > 0)
24             cout << re << " + i" << fabs(im) << endl;
25         else
26             cout << re << " - i" << fabs(im) << endl;
27     }
28
29     void subCom(Complex c1, Complex c2);
30     void addCom(Complex c1, Complex c2);
31     void mulCom(Complex c1, Complex c2);
32     void divCom(Complex c1, Complex c2);
33
34     void Complex::subCom(Complex c1, Complex c2)
35     {
36         re = c1.re - c2.re;
37         im = c1.im - c2.im;
38     }
39
40     void Complex::addCom(Complex c1, Complex c2)
41     {
42         re = c1.re + c2.re;
43         im = c1.im + c2.im;
44     }
45
46     void Complex::mulCom(Complex c1, Complex c2)
47     {
48         re = (c1.re * c2.re) - (c1.im * c2.im);
49         im = (c1.re * c2.im) + (c1.im * c2.re);
50
51     void Complex::divCom(Complex c1, Complex c2)
52     {
53         double denominator = (c2.re * c2.re) + (c2.im * c2.im);
54         if (denominator == 0)
55             cout << "Error: Division by zero complex number!" << endl;
56         re = ((c1.re * c2.re) + (c1.im * c2.im)) / denominator;
57         im = ((c1.im * c2.re) - (c1.re * c2.im)) / denominator;
58     }
59
60     int main()
61     {
62         Complex c1, c2, c3;
63         cout << "Enter first complex number:" << endl;
64         c1.input();
65         c1.show();
66         cout << "\nEnter second complex number:" << endl;
67         c2.input();
68         c2.show();
69         cout << "\n--- Results ---" << endl;
70         c3.addCom(c1, c2);
71         cout << "Sum: ";
72         c3.show();
73         c3.subCom(c1, c2);
74         cout << "Subtraction: ";
75         c3.show();
76         c3.mulCom(c1, c2);
77         cout << "Multiplication: ";
78         c3.show();
79         c3.divCom(c1, c2);
80         cout << "Division: ";
81         c3.show();
82         return 0;
83     }
```

Output:

```
Enter first complex number:
Enter real part: 3
Enter imaginary part: 5
3 + i5

Enter second complex number:
Enter real part: 4
Enter imaginary part: 7
4 + i7

--- Results ---
Sum: 7 + i12
Subtraction: -1 - i2
Multiplication: -23 + i41
Division: 0.723077 - i0.0153846
```